DeltaCycle: A Multi-Scale Linear-Attention Network with a Physics-Consistent Degradation-Rate Head for Battery Prognostics

Naiming Xie[a,*], Yuan Wang[a], Mingshan Li[a]

[a] *College of Economics and Management, Nanjing University of Aeronautics and Astronautics, 29 Jiangjun Avenue, Nanjing, 211106, China*

Abstract

Lithium-ion battery fleets—in electric vehicles, robotics, portable medical devices, and second-life markets—generate a continuous stream of maintenance and replacement decisions, in a market projected to grow several-fold over the coming decade. Every such decision requires three things that current monitoring technology cannot deliver simultaneously: models accurate enough to be decision-grade, yet small enough to run on the microcontrollers inside battery management systems rather than in the cloud; forecasts that stay trustworthy as horizons lengthen and field data corrupts; and uncertainty statements that let operators trade risk against cost explicitly.

We propose DeltaCycle, a capacity-only battery monitoring loop built for deployment. Its multi-scale linear-attention encoder is deployment-verified via a simplest-representative single-branch variant that runs in C on a microcontroller with a fixed-size recurrent state, matching PyTorch within floating-point rounding; a physics-consistent degradation-rate head driven by measured internal resistance—the macroscopic signature of lithium-inventory loss, brought into the learning loop as a constraint on the decay direction; and conformal-calibrated prediction intervals that convert point forecasts into decision-grade outputs.

Across five public datasets spanning four chemistries, DeltaCycle achieves first-tier accuracy among protocol-matched baselines—state of the art on PANASONIC, matched against the strongest published explicit tracking of

*Corresponding author

Email address: xienaiming@nuaa.edu.cn (Naiming Xie)

capacity regeneration. The physics term is validated where physics should matter: it doubles unseen-tail extrapolation fidelity and wins every corruption-robustness setting tested—direct evidence that the mechanism earns its place. Interval coverage is lifted to 93.3% at nominal 95%. For battery operators this translates into a capability shift: continuous, risk-aware asset monitoring that runs on-device at near-zero marginal cost, without cloud connectivity, model retraining, or specialist hardware.

Keywords: Battery RUL, SOH prediction, Predictive maintenance, Linear attention, Gated DeltaNet, Edge deployment, Uncertainty quantification

---

1. Introduction

Lithium-ion batteries power electric vehicles, grid-scale energy storage, spacecraft, and portable electronics, and the market they serve is projected to grow several-fold over the coming decade [1, 2]—making battery health management a first-order economic concern rather than a technical nicety. Each application stresses battery management differently: electric-vehicle packs must track health under rapidly varying current, temperature, and state of charge, where capacity fade shortens range and raises thermal-safety risk; grid-scale storage integrates hundreds of thousands of cells into hierarchical cell–pack–cluster–container systems, where single-cell aging accumulates into fleet-level inconsistency that dominates life-cycle operating cost [3]; spacecraft batteries must operate for years without on-site repair under radiation, vacuum, and extreme temperatures, where power-subsystem anomalies are a leading cause of mission loss [2]; and portable electronics make battery health the most ubiquitous—and least supervised—monitoring problem of all.

Battery management systems (BMS) distill battery health into three state quantities: *state of charge* (SOC), the fraction of remaining usable charge—the "fuel gauge"; *state of health* (SOH), the current full capacity relative to rated capacity—the "health report," whose convention-specified end-of-life threshold (typically 70–80% of rated capacity) defines retirement; and *remaining useful life* (RUL), the number of cycles until that threshold is crossed—the "countdown." The three form a progression: SOC describes how much energy remains now, SOH how far aging has progressed, and RUL how long the asset will keep serving; of the three, RUL is the forward-looking quantity that maintenance and replacement decisions actually consume [1, 2].

That progression rests on a fundamental contradiction: the chemistry prized for its high energy density and long cycle life ages inevitably under use. Two microscopic mechanisms dominate the aging—*loss of lithium inventory*, as the growing solid-electrolyte interface consumes cyclable lithium, and *loss of active material*, as electrode particles crack and detach—and both manifest macroscopically as capacity fade and rising internal resistance [1]. The resulting degradation is inherently nonlinear and heterogeneous: a long-term monotonic fade is intertwined with capacity regeneration events, stage-wise transitions, and measurement noise, making RUL prediction a challenging prognostics problem [4, 5]. The battery's internal state is moreover a black box: health indicators must be inferred from external measurements rather than observed directly, which is why health prediction is inherently uncertain—and why an estimator that consumes only field-measurable quantities has direct operational value.

For operators, RUL is far more than an academic metric. It converts maintenance from reactive repair to planned intervention; it informs retirement and second-life decisions that extend a battery's economic life; it bounds safety margins, since capacity fade and resistance rise are recognized precursors of thermal runaway; and it caps life-cycle cost, the dominant expense of large storage fleets [1, 3].

Existing RUL prediction methods fall into two broad families. Model-based approaches—equivalent-circuit models, extended and unscented Kalman filters, and particle filters—encode degradation physics explicitly [6, 7, 8, 9, 10] but require accurate parameter identification and often lack the flexibility to capture regeneration dynamics. Data-driven approaches learn degradation patterns directly from cycling data. Within this family, recurrent networks (LSTM, GRU) [11, 12, 13] model sequential capacity evolution but suffer from long-term dependency limitations [4]; Transformer-based models [14] such as PatchFormer [15] and general time-series transformers [16] [17] achieve strong accuracy but their quadratic attention complexity and growing memory footprint hinder deployment on resource-constrained embedded platforms. Most recently, state space models (SSMs) such as Mamba [18] have been applied to battery prognostics, with RUL-Mamba [4] reporting state-of-the-art performance on NASA, Oxford, and TJU datasets—yet the reported efficiency is measured on GPUs, and edge-deployment feasibility is not addressed.

Despite these advances, four limitations remain. First, quadratic self-attention makes Transformer-based models difficult to deploy on microcontrollers (MCUs) used in BMS: the attention score matrix grows with sequence

length, requiring dynamic memory allocation that is undesirable in safety-critical firmware. Second, degradation information lives at multiple time scales—fast regeneration events, intermediate stage transitions, and the slow monotonic fade—and models that treat the capacity sequence as a single scale either over-smooth the recoveries or lose the global trend [5]. Third, purely data-driven models are evaluated on clean laboratory data and falter exactly where field operation deviates from the lab: the end-of-life acceleration tail—where maintenance decisions concentrate—is the regime with the least training data, so learned predictors flatten or drift there, and corrupted or noisy sensor streams sharply degrade their predictions. Fourth, point estimates of SOH and RUL do not convey uncertainty, which limits their use in risk-aware BMS decisions.

Beneath these technical gaps lies a decision-layer gap: even an accurate SOH/RUL estimate is an intermediate product, and translating it into maintenance actions today still relies on expert judgment. Field evidence from large-scale storage systems shows that explainable, decision-oriented operation and maintenance frameworks cut response time and operational cost by over 80% compared with conventional expert-driven practice [3]. The extreme operating regime is space: the vast majority of new satellites run on lithium-ion batteries, in-orbit repair is impossible, and power-subsystem anomalies are a leading cause of satellite faults [2]—an environment where on-device, decision-grade prognostics is a requirement rather than a convenience.

This paper proposes *DeltaCycle*, a multi-scale linear-attention network for battery prognostics designed from the outset for embedded deployment. Our contributions are:

1. **Linear-attention backbone with verified edge deployment.** We adopt Gated DeltaNet-2 (GDN-2), a recurrent linear attention layer with a fixed-size matrix state (8 KB per layer in the deployed configuration), and ship a full C implementation that matches the PyTorch forward within floating-point rounding, with arithmetic and libm components verified bit-identical across x86 and ARM emulation. *Operationally, the accuracy machinery is no longer tied to the cloud: the model runs inside the BMS microcontroller itself—the prerequisite for per-device monitoring at fleet scale and near-zero marginal cost.*
2. **Multi-scale branches with stage-query cross-scale exchange.** Three patch branches (sizes 2/4/8) read degradation at local, regional,

and global scales; after every layer the coarse branch's GDN state—the compressed degradation history—projects to a single query that reads the fine and mid branch sequences. Ablations show the exchange is essential. *Operationally, the coarse branch acts as an explicit degradation-stage indicator—the piece of information maintenance scheduling actually consumes.*

3. **A physics-consistent degradation-rate head.** Why physics at all? The third limitation above is the motivation: learned predictors flatten or drift exactly where the data is thinnest, and physics supplies an inductive bias that remains valid outside the observed distribution—capacity decay is monotone, and internal resistance can only accelerate it. The challenge is injecting that bias without breaking the standard evaluation protocol. A systematic injection-route study shows why conventional physics injection fails under that protocol (the z-score target space is structurally incompatible with monotone predictors), and yields the mechanism that *does* work: a rate head driven by measured internal resistance, constrained so that resistance can only accelerate decay, trained in absolute capacity space. The contribution is the physics itself, and its value is demonstrated where physics should matter: unseen-tail extrapolation fidelity doubles and the head wins every corruption-robustness setting tested—evidence that the mechanism is doing measurable work rather than acting as a decorative constraint.
4. **Decision-grade uncertainty at one forward pass.** A single pinball-trained model emits P2.5/P50/P97.5 intervals; conformal calibration on a held-out cell lifts coverage to 93.3% at nominal 95%, with two scalars and no ensemble. *The interval, not the point estimate, is what a replacement policy consumes: the calibrated band bounds the missed-EOL rate that a risk-averse scheduler actually cares about.*

The technical contributions above translate into an operational value chain in which each application stresses a specific DeltaCycle capability rather than generic monitoring (Figure 1): no-connectivity edge deployment (spacecraft satellites and UAV fleets, where physical access and onboard connectivity budget rule out cloud dependence), risk-aware scheduling with calibrated intervals (energy-storage battery cabinets and power-battery packs, where the missed-EOL rate sets the maintenance window), and regeneration tracking for non-monotonic field histories. We stress that the experimental validation in this paper is conducted on public laboratory cycling data; the

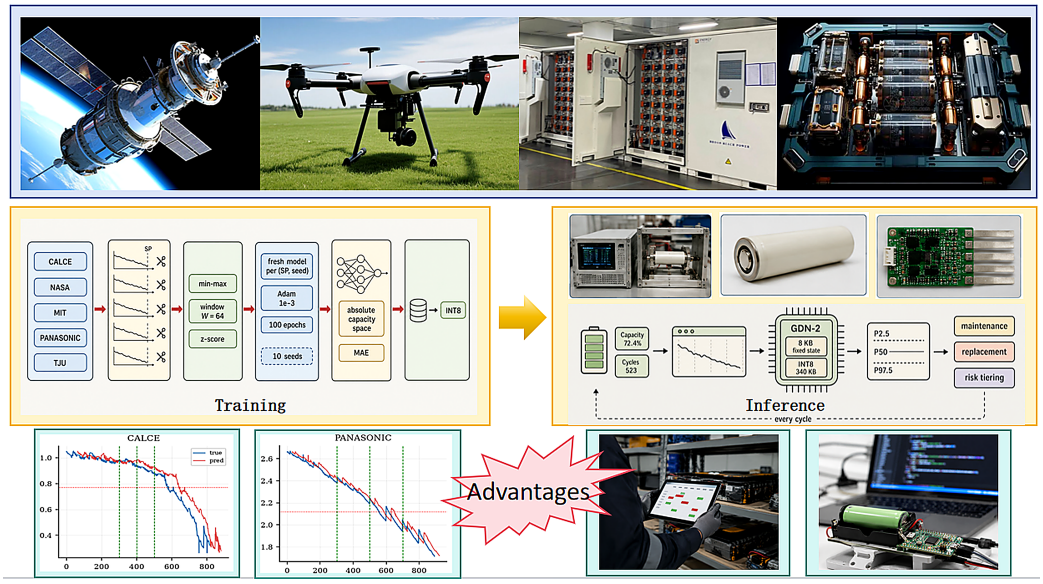


Figure 1: System overview: the overall pipeline of the proposed method and an outlook on its application across physical battery systems with different power and operating profiles—spacecraft satellites, unmanned aerial vehicles (UAVs), energy-storage battery cabinets, and power-battery packs. The pipeline starts from public cycling datasets (CALCE, NASA, MIT, PANASONIC, TJU), performs data preprocessing and trains a fresh model per (dataset, starting point, seed) under the PatchFormer-consistent protocol, and exports an INT8-quantized model whose fixed 8 KB state and minimal compute footprint make it deployable on resource-constrained embedded devices, laying the foundation for on-device online monitoring. Application images are illustrative; the experimental validation is conducted on public laboratory data.

figure is an *application outlook*, not a deployment report.

The remainder of this paper is organized as follows. Section 2 reviews related work. Section 3 describes the proposed architecture. Section 4 presents datasets, evaluation protocols, and experimental results, including ablations, comparison with PatchFormer, and deployment validation. Section 5 concludes the paper.

2. Related Work

2.1. Model-based and traditional machine-learning methods

Physics-based prognostics build degradation models from electrochemical dynamics, estimated with filters such as extended Kalman filters [6], unscented particle filters [7], and particle filters with electrochemical or empirical state models [8, 9, 10]. These methods encode mechanistic knowledge but require accurate parameter identification and struggle with regeneration-rich or heterogeneous degradation. Traditional machine learning provides a lighter alternative: support vector machines [19, 20], support vector regression [21], and relevance vector machines [22] regress health indicators directly from capacity or voltage features, yet their fixed kernels cannot easily extrapolate over the strongly nonlinear EOL-acceleration phase.

2.2. Deep sequence models for battery prognostics

Recurrent models were among the first deep approaches to capacity sequences: LSTM-based predictors [12, 23], GRU variants [13, 24], and recurrent networks for RUL [25] capture temporal dynamics but suffer from long-term dependency limitations. Convolutional hybrids combine local feature extraction with recurrence, including CNN-BiLSTM [26], TCN-LSTM [27], and attention mechanisms [11]. Transformer-based models [14] brought global attention to battery prognostics: direct Transformer RUL predictors [28], hybrid CEEMDAN-Transformer-DNN models [29], and PatchFormer [15], which combines dual patch-wise attention with feature-wise attention. General time-series transformers evaluated on battery tasks include Autoformer [16], FEDformer [17], iTransformer [30], PatchTST [31], TimesNet [32], and TimeMixer [33]. All attention-based methods share the quadratic-complexity bottleneck that constrains embedded deployment. Comprehensive surveys of this line are given in [34, 35].

2.3. State space models

Mamba [18] is a selective state space model with linear-time inference, extensively surveyed in [36] and evaluated for general time-series forecasting in [37]. For batteries, MambaLithium [38] applies a pure Mamba network to SOH/RUL/SOC estimation, and RUL-Mamba [4] combines a Mamba encoder with a feature attention network and a gated residual decoder, reporting state-of-the-art results on NASA, Oxford, and TJU datasets. While these works emphasize efficient training and fast inference, their efficiency claims are based on GPU measurements; none of them validates execution on embedded hardware, which is the target of the present work.

2.4. Grey system models

Grey system theory models degradation using accumulated sequences and fractional-order operators. Recent work includes grey models with capacity-regeneration consideration [39], adaptive weighted fractional-order reverse accumulation for SOH [40], fractional-order heuristic models for PEMFC prognostics [41], self-adaptive grey neural networks for real-time PEMFC degradation [42], grey Kalman filtering for SOC estimation [43], and grey iterative modeling for battery health management [44]. These methods are lightweight and interpretable but rely on hand-crafted model forms that must be re-specified for each battery chemistry and operating regime.

2.5. Physics-informed battery modeling

Physics-informed neural networks (PINNs) embed degradation physics into the learning objective. Wang et al. [45] proposed a physics-informed network for stable degradation modeling, and PiDDM [46] incorporates Arrhenius degradation kinetics into the training objective, reporting improved extrapolation at the end of life. Our systematic injection-route study (Section 4.4) revisits this paradigm for a sequence model that already ingests the capacity trajectory: objective regularization, input feature concatenation, and structural heads all fail—the per-window z-score target space is structurally incompatible with monotone predictors, and the features are redundant with capacity history. The mechanism that survives is a degradation-rate head driven by measured internal resistance, trained in absolute capacity space (Section 3.4).

2.6. Linear attention and recurrent mixing

Linear attention replaces the softmax attention map with a recurrent matrix state, achieving $O(N)$ complexity. Gated DeltaNet [47] combined the delta rule with learned gating, and Gated DeltaNet-2 (GDN-2) [48] decoupled channel-wise erase and write gates, improving long-context retrieval in language modeling. To our knowledge, the present work is the first to report floating-point-accurate MCU deployment of a linear-attention battery model.

Stepping back from individual architectures, the surveyed literature answers one question—which model is most accurate on public cycling data—while leaving the questions that decide operational adoption largely open. First, accuracy has been demonstrated only on server-class hardware: none of the battery-specific models above reports a deployment path onto the microcontrollers that production BMS firmware actually runs on, so operators must either pay for cloud connectivity or accept simpler on-device estimators. Second, forecasts are evaluated as point trajectories, yet the operational consumer of a forecast is a maintenance schedule, which requires an explicit statement of risk; few works report calibrated intervals suitable for scheduling. Third, evaluations are confined to clean laboratory data: the end-of-life acceleration tail and the corrupted-sensor regimes—where field decisions concentrate—are rarely tested, so reported accuracy says little about how a predictor will behave in deployment. DeltaCycle is designed around these three operational gaps rather than around a single benchmark number.

3. Method

3.1. Problem formulation and input design

We formulate battery prognostics as a sliding-window capacity forecasting problem. Let c_t denote the measured capacity at cycle t. Given a fixed-length input window $\mathbf{X}_t = [c_{t-W+1}, \ldots, c_t]$ of historical capacity, the model predicts future capacity values. The main model deliberately consumes *capacity only*, for a deployment-driven reason: at inference, the future values of voltage, current, temperature, and internal resistance are unavailable, so any model consuming them cannot be rolled out autoregressively to the end of life without information leakage. Capacity is the only health indicator that is both measurable online and self-contained for rollout. The physics-consistent extension of Section 3.4 additionally consumes the per-cycle *measured* internal

resistance—a quantity the BMS records in situ, available at inference without future information—so the extension remains deployable under the same constraint.

3.2. Gated DeltaNet-2 backbone

The backbone is Gated DeltaNet-2 (GDN-2) [48], a recurrent linear attention layer with channel-wise erase and write gates. For one head with key dimension d_k and value dimension d_v, the recurrence is

$$\mathbf{S}_t \;=\; \big(\mathbf{I} - \mathbf{k}_t(\mathbf{b}_t \odot \mathbf{k}_t)^\top\big)\,\mathrm{diag}(\boldsymbol{\alpha}_t)\,\mathbf{S}_{t-1} \;+\; \mathbf{k}_t(\mathbf{w}_t \odot \mathbf{v}_t)^\top, \tag{1}$$

$$\mathbf{o}_t \;=\; \mathbf{S}_t^\top \mathbf{q}_t, \tag{2}$$

where $\mathbf{q}_t, \mathbf{k}_t \in \mathbb{R}^{d_k}$ and $\mathbf{v}_t \in \mathbb{R}^{d_v}$ are query, key, and value vectors; $\mathbf{b}_t \in \mathbb{R}^{d_k}$ is the channel-wise erase gate and $\mathbf{w}_t \in \mathbb{R}^{d_v}$ the channel-wise write gate, both bounded to $[0, 1]$ by a sigmoid; and $\boldsymbol{\alpha}_t = \exp(\mathbf{g}_t)$ is the channel-wise decay with

$$\mathbf{g}_t \;=\; -A_{\log} \cdot \mathrm{softplus}\big(f(\mathbf{x}_t) + \mathbf{d}_t\big), \tag{3}$$

where $\mathbf{x}_t$ is the input token at position t (the embedded capacity, or the branch's hidden representation after the first layer), $A_{\log}$ is a per-head log-rate, $f(\cdot)$ a two-layer projection, and $\mathbf{d}_t$ a per-channel bias. The state $\mathbf{S}_t \in \mathbb{R}^{d_k \times d_v}$ is fixed-size and updated incrementally, yielding $O(N)$ scan complexity with constant memory. With H heads, $d_k = 16$, and $d_v = 32$, the total state is $H \cdot d_k \cdot d_v \cdot 4$ bytes $= 8\,\mathrm{KB}$ per layer. Figure b contrasts the per-window cost of this scan with quadratic self-attention. During a window scan the state is updated incrementally—each new cycle erases part of the old memory (via $\mathbf{b}_t$) and writes new information (via $\mathbf{w}_t$)—so the fixed-size matrix acts as an adaptive compression of the degradation history rather than a static buffer.

3.3. Multi-scale branches with per-layer cross-scale exchange

Figure 2 shows the multi-scale encoder together with the per-layer cross-scale exchange submodule (bottom, light-green box), and Figure 3 details the internal recurrence of one GDN-2 block.

Battery degradation operates across multiple temporal scales: short regeneration events, mid-range knee-point transitions, and the long-term monotonic envelope. We encode these with three parallel branches using patch sizes $P \in \{2, 4, 8\}$ (Figure c shows the real capacity series at each patch

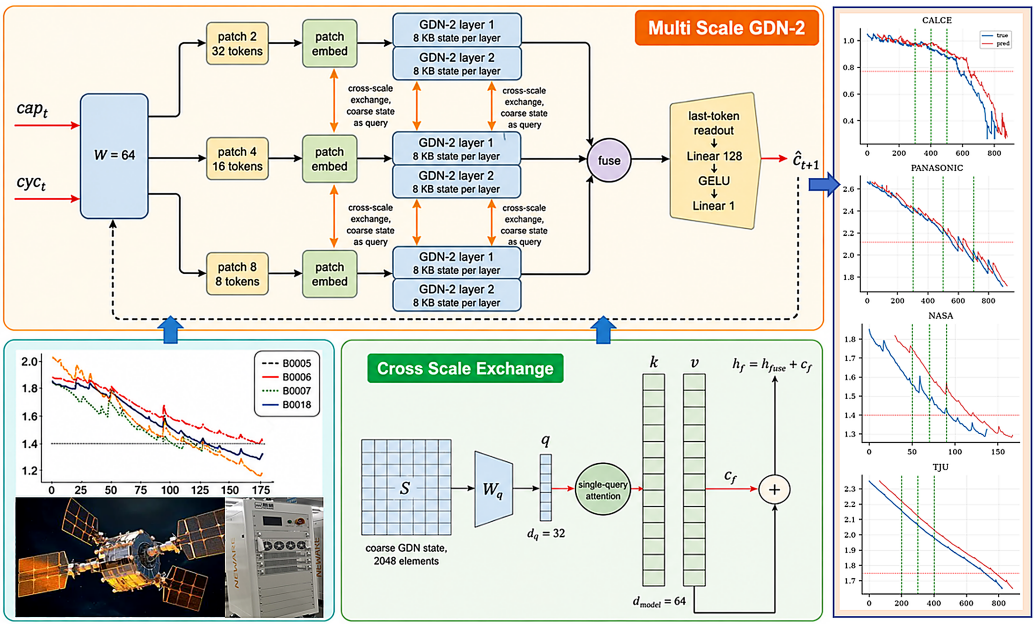


Figure 2: Multi-scale GDN-2 architecture and cross-scale exchange submodule. Top: the fixed window of W=64 capacity cycles (the tokens cap_t; cyc_t denotes the cycle index) is encoded in parallel at three scales (patch 2/4/8, 32/16/8 tokens), embedded per scale, and passed through two GDN-2 layers per branch (8 KB fixed state per layer); the three branches exchange multi-scale information through the per-layer cross-scale exchange (orange arrows, coarse state as query), fuse at the $fuse$ node, and emit the next-cycle capacity prediction $\hat{c}_{t+1}$ via the last-token readout (Linear 128 $\rightarrow$ GELU $\rightarrow$ Linear 1). Bottom (light-green box): the cross-scale exchange submodule maps the coarse state S (2048 elements) to a single query q (d_q=32) via W_q; single-query attention interacts q with the fine/mid branch keys and values (k, v, d_{model}=64) to produce the context c_f, added to the fused state: $h_f = h_{\text{fuse}} + c_f$.

scale). Each branch projects its patched input to a common $d_{\text{model}} = 64$ embedding and applies two GDN-2 layers. After every GDN layer, branches exchange information through a stage-query mechanism: the coarse branch's GDN-2 state—the compressed degradation history, which acts as a stage indicator—is projected to a single query, and the fine and mid branch sequences provide keys and values. For branch $\mathbf{h}_a$ with the coarse state $\mathbf{S}_c$,

$$\mathbf{q} = \mathbf{W}_q \,\mathrm{vec}(\mathbf{S}_c), \qquad \mathbf{h}'_a = \mathbf{h}_a + \sum_t \frac{\exp\!\left(\mathbf{k}_t^\top \mathbf{q}/\sqrt{d_k}\right)}{\sum_s \exp\!\left(\mathbf{k}_s^\top \mathbf{q}/\sqrt{d_k}\right)} \mathbf{v}_t, \tag{4}$$

where $\mathbf{k}_t = \mathbf{W}_k \mathbf{h}_a[t]$ and $\mathbf{v}_t = \mathbf{W}_v \mathbf{h}_a[t]$, and the summation is a single-

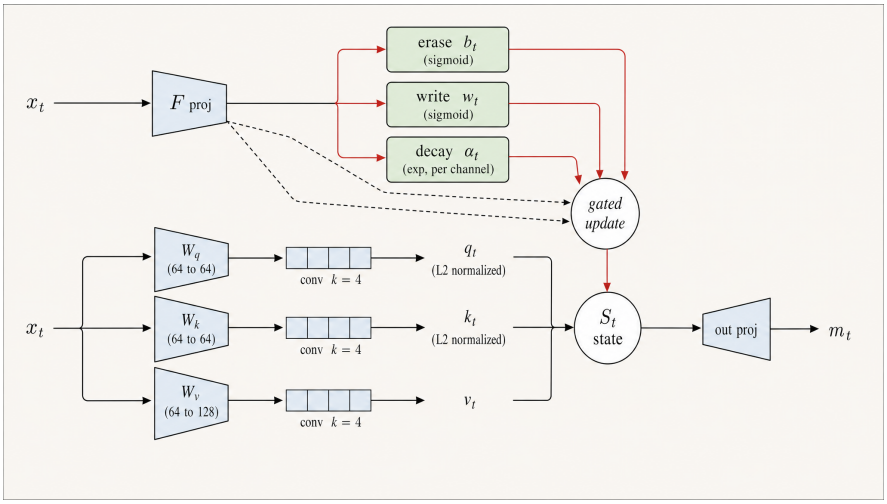
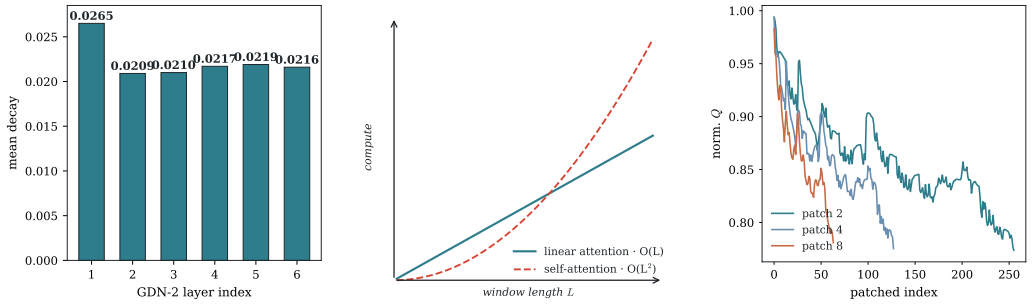


Figure 3: One GDN-2 block. The input token x_t is projected to queries/keys/values (W_q: $64 \to 64$, W_k: $64 \to 64$, W_v: $64 \to 128$), each through a causal 1-D convolution (kernel k=4, $L2$-normalized for q and k), while a shared projection F_{proj} generates the erase ($\mathbf{b}_t$, sigmoid), write ($\mathbf{w}_t$, sigmoid) and per-channel decay ($\boldsymbol{\alpha}_t = \exp(g)$) gates; the gated update produces the fixed-size state $\mathbf{S}_t$ ($4 \times 16 \times 32$ elements = 8 KB, zero growth), and the output $\mathbf{o}_t = \mathbf{S}_t^\top \mathbf{q}_t \to$ Linear $128 \to 64 \to$ RMSNorm.



(a) (b) (c)

Figure 4: Method-consistency evidence. (a) Real per-layer mean decay weights from the seed-42 CALCE checkpoint: each layer keeps state over hundreds of cycles, and the per-channel log-rate gate ($g = -\exp(A_{\log})\,\text{softplus}(\text{raw}_g + \text{dt}_{\text{bias}})$, $\boldsymbol{\alpha}_t = \exp(g)$) sets how quickly each channel forgets. (b) Per-window cost of the single-query linear-attention scan ($O(L)$) versus quadratic self-attention ($O(L^2)$). (c) Real multi-scale capacity series of the W=64 CALCE window under patch sizes $P \in \{2, 4, 8\}$ (32/16/8 tokens).

query linear-attention readout with $O(L)$ cost per layer. The exchange is additive (residual, no gating) and applied per layer. Ablation studies (Section

4.4) show that branches without interaction degrade consistently (CALCE MAE 0.0068 vs 0.0065; Wilcoxon $p = 0.037$), while the stage-query exchange restores the multi-scale model to the single-branch level on CALCE and yields the best mean on the regeneration-rich NASA dataset (0.0084 vs 0.0087); the exchange never degrades accuracy relative to a single branch.

An interpretability analysis (Section 4.5) reveals that the coarse branch (patch 8) spontaneously encodes the degradation stage: its pooled representation is 93.2% linearly separable into early/mid/late life stages on the PANASONIC dataset (2.9× chance), providing stage-aware state awareness at no additional supervision cost.

3.4. Physics-consistent extension: the rate head

The main architecture of Section 3.3 is trained purely data-driven under the baseline-consistent per-window protocol. Here we present a physics-consistent extension whose design is the outcome of a systematic injection-route study (Section 4.4): a degradation-rate head driven by a measured physical feature. Lithium-ion capacity fade is governed by thermally activated processes—solid-electrolyte-interface growth, loss of lithium inventory, and active-material loss—and internal resistance (IR) is a direct, independently measured observable of these aging processes that rises as the cell degrades. We therefore decompose the per-cycle decay rate into a learned component and a physics-feature component:

$$r \;=\; \mathrm{softplus}(\mathbf{w}^{\top}\mathbf{h}) \;+\; \mathrm{softplus}(\gamma)\cdot \mathrm{IR}_{\mathrm{last}}, \qquad \hat{c}_{t+1} \;=\; c_t - r, \tag{5}$$

where $\mathbf{h}$ is the last-token GDN state, c_t the current capacity, and $\mathrm{IR}_{\mathrm{last}}$ the measured internal resistance at the current cycle. The softplus constraint makes $\gamma \geq 0$: internal-resistance growth can only *accelerate* decay, never decelerate it—the physical direction of SEI-driven aging. The formulation is structurally monotonic ($\hat{c}_{t+1} \leq c_t$) and therefore cannot emit nonphysical regeneration artifacts by construction.

The rate head is trained in *absolute* capacity space ($\mathcal{L} = \mathrm{MAE}(\hat{c}, c)$) rather than the per-window z-score protocol: z-score targets include positive values (plateau segments where the next observation lies above the window mean) that a monotone predictor can never produce, so the constraint is unsatisfiable in that space; the absolute-space objective removes the sign conflict. A control experiment confirms the mechanism, not the objective, is responsible for the gain: a free head trained in absolute space reaches

MAE 0.0097 (worse than the z-score free head at 0.0070), while the rate head reaches 0.0042. The extension further improves the unseen-tail extrapolation (R^2 0.775 vs 0.374) and robustness to corrupted inputs (Section 4.4).

3.5. Readout and training protocol

After the multi-scale encoder, the fused last-token representation $\mathbf{h}_t$ is passed through a readout

$$\hat{c}_{t+1} \;=\; \mathrm{Head}(\mathbf{h}_t) \;=\; \mathbf{W}_2\,\mathrm{GELU}(\mathbf{W}_1\mathbf{h}_t), \tag{6}$$

with $d = 3 \cdot d_{\text{model}} = 192$ the dimension of the fused last-token representation (concatenation of the three branches), $\mathbf{W}_1 \in \mathbb{R}^{128\times d}$ and $\mathbf{W}_2 \in \mathbb{R}^{1\times 128}$. The model is trained with the per-window z-score target protocol of PatchFormer and RUL-Mamba for baseline comparability: the input window is normalized globally with training-cell min–max statistics, the target is $(c_{t+1}-\bar{c})/\sigma_c$ with window statistics $(\bar{c}, \sigma_c)$, and predictions are de-normalized with the same window statistics at evaluation.

3.6. Uncertainty quantification with conformal calibration

Safety-critical BMS decisions require not only a point estimate of the health state but a calibrated interval, e.g., "the cell will cross the EOL threshold within 30–50 cycles with 95% confidence". To provide this without the cost of an ensemble, the readout head outputs three quantiles per forward pass, $q \in \{0.025, 0.5, 0.975\}$ (P2.5, the median P50, and P97.5), bounding a central 95% interval. The head is trained with the pinball (quantile) loss

$$\mathcal{L}_{\text{pin}} \;=\; \frac{1}{N}\sum_{i}\;\sum_{q\in\{0.025,0.5,0.975\}} \rho_q\Big(c_i - \hat{c}_i^{(q)}\Big), \qquad \rho_q(u) = \max(qu, (q-1)u), \tag{7}$$

where c_i is the observed capacity, $\hat{c}_i^{(q)}$ is the predicted q-quantile, and the pinball function ρ_q penalizes under-prediction (when $\hat{c}^{(q)} > c$) by a factor q and over-prediction by $1-q$, so that the minimizer is the conditional q-quantile. Because a lower capacity trajectory crosses the EOL threshold earlier, P2.5 yields the *earliest* plausible EOL crossing and P97.5 the *latest*; the P2.5–P97.5 crossing window is therefore a safety-relevant interval (the pessimistic bound drives maintenance scheduling).

Quantile regression alone is known to be miscalibrated under distribution shift. We therefore apply *conformalized quantile regression* (CQR): a held-out calibration cell provides conformity scores $s_i = \max(\hat{c}_i^{\text{lo}} - c_i,\; c_i - \hat{c}_i^{\text{hi}})$, and

the deployed interval is inflated by q_{adj}, the $(1-\alpha)(1+1/n)$ empirical quantile of the scores, where $\alpha = 0.05$ and n is the calibration size. Calibration and coverage results are reported in Section 4.6. The entire UQ mechanism costs one forward pass and two scalars at deployment—no ensemble, no MC sampling.

4. Experiments

4.1. Datasets

We evaluate on five public battery degradation datasets spanning different chemistries, capacities, and degradation behaviors (Table 1): CALCE (LCO, rated 1.1 Ah), NASA [49] (LCO, 2.0 Ah), MIT/Stanford [50] (LFP, 124 cells), PANASONIC (NCA, 3.03 Ah), and TJU (NCM+NCA, 2.5 Ah). End-of-life (EOL) thresholds follow rated capacity times convention: 70% for small cells and 80% for large-format cells. For MIT we use a 10-cell subset (8 training, 2 test) for this study. All models use pure capacity input; internal resistance is used only as an auxiliary input of the physics-extension rate head (Section 3.4).

Table 1: Dataset overview.

Dataset	Chemistry	Rated	Cells	EOL (Ah)	Role
CALCE	LCO	1.1 Ah	4	0.77	primary, physics (IR)
NASA	LCO	2.0 Ah	4	1.40	short-history
MIT	LFP	1.07 Ah	10*	0.86	cross-battery
PANASONIC	NCA	3.03 Ah	3	2.12	regeneration
TJU	NCM+NCA	2.5 Ah	3	1.75	long-cycle

4.2. Evaluation protocol

Following the standard protocol in PatchFormer [15] and RUL-Mamba [4], we report trajectory metrics per starting point (SP): R^2, MAE (mean absolute error), RMSE (root mean squared error), and AE. Let TRUL be the true remaining useful life in cycles at an SP (cycles from the SP to the true EOL crossing), PRUL the predicted RUL from the first threshold crossing of the predicted capacity sequence, $\text{AE} = |\text{TRUL} - \text{PRUL}|$ the absolute RUL error in cycles, and $\text{RE} = \text{AE}/\text{TRUL}$ the relative RUL error.

Training configuration. All models use Adam (learning rate 1×10^{-3}, batch size 64) for 100 epochs with a window of W=64 cycles (W=30 for NASA and PANASONIC, matching the window convention of the baseline literature). These settings were fixed during the ablation stage and held constant across datasets, seeds, and model variants—no per-dataset or per-variant tuning is applied to the reported results, so every variant compared in Sections 4.4–4.5 shares the same optimizer budget. We do not claim these configurations are optimal; a sensitivity note is included in Section 4.8. The main results (Table 2) additionally follow the per-SP training protocol of PatchFormer: for each (dataset, SP, seed) a fresh model is trained from scratch, on the non-test cells' full sequences plus the test cell's cycles before the SP, and evaluated on the segment starting at the SP; we use ten seeds per SP (1–10), so each table cell aggregates ten independently trained models. The ablation studies (Section 4.4) use the same optimizer budget with ten seeds; the physics extension is reported over the same ten seeds for its head, extrapolation, and robustness studies, and the uncertainty study uses the single-seed, full-data training described above (variance quantified in Section 4.8).

4.3. Main results

Figure 5 shows representative SOH trajectories with sliding-window single-step predictions for one cell of each dataset; Table 2 reports the full per-SP results. The five datasets cover distinct degradation regimes and operational scenarios; we present them as case studies below, each closing with its operational implication and a comparison with published results on that dataset.

Battery-specific foundation and pretrained models published outside the per-SP RUL protocol family are not included in the comparison tables, because their reported metrics use different evaluation conventions (early-cycle prediction, or trajectory regression without per-SP RUL errors); head-to-head comparisons under our protocol are left to future work. Each case study below carries the per-dataset comparison table against PatchFormer [15], RUL-Mamba [4], and the general time-series transformers, using the numbers reported in those papers on the shared test cells, EOL thresholds, and starting points (NASA B0005 and TJU CY25-1 are used identically by all works). AE is filled wherever the source paper reports it: RUL-Mamba's NASA and TJU rows carry its reported AE (an average over 10 runs), and on CALCE and PANASONIC—where RUL-Mamba's own paper has no results—we use OmniTIEFormer's baseline runs of RUL-Mamba [5], which also report AE.

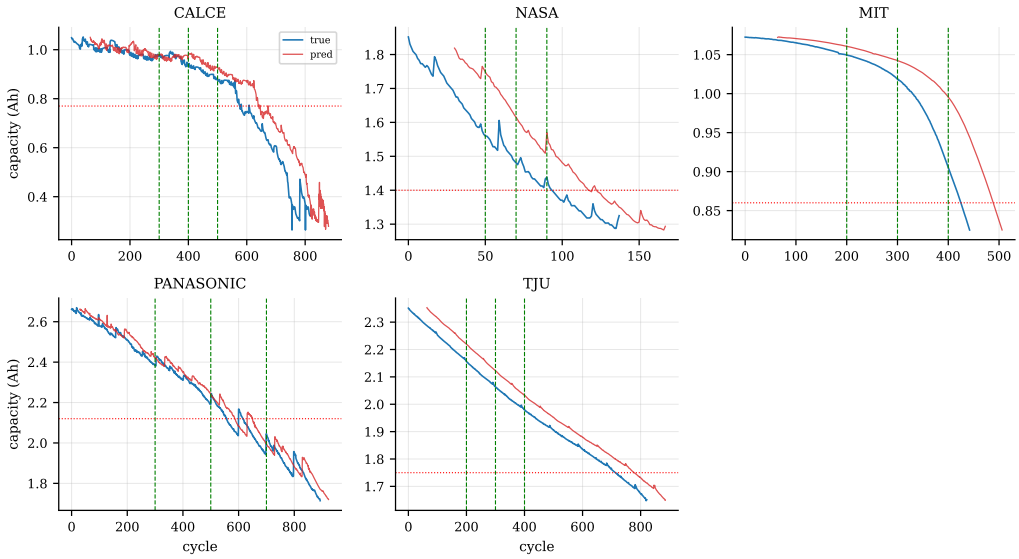


Figure 5: Single-step capacity prediction on one test cell per dataset. Green dashed lines mark the starting points (SPs); red dotted lines mark the EOL thresholds.

Table 2: Per-SP trajectory results, mean ± std over ten seeds (1–10) per SP under the PatchFormer-consistent per-SP protocol (a fresh model per (SP, seed)); MIT rows report the full-data model of Sec. 4.3). TRUL in cycles; MAE/RMSE are trajectory errors in normalized capacity; AE (a small bounded integer) is reported as mean over seeds.

Dataset	SP	TRUL	MAE	RMSE	R^2	AE
CALCE	300	339	0.0066±0.0007	0.0146±0.0013	0.9951±0.0008	3.2
	400	239	0.0078±0.0010	0.0170±0.0019	0.9933±0.0015	2.5
	500	139	0.0092±0.0016	0.0196±0.0028	0.9898±0.0030	2.4
NASA	50	73	0.0098±0.0018	0.0145±0.0013	0.9908±0.0018	0.9
	70	53	0.0089±0.0009	0.0148±0.0005	0.9822±0.0011	0.4
	90	33	0.0080±0.0009	0.0115±0.0011	0.9804±0.0036	0.5
PANASONIC	300	287	0.0037±0.0003	0.0096±0.0001	0.9976±0.0001	0.8
	400	187	0.0036±0.0002	0.0101±0.0001	0.9965±0.0001	0.8
	500	87	0.0040±0.0003	0.0111±0.0003	0.9936±0.0003	0.9
TJU	200	577	0.0013±0.0002	0.0020±0.0001	0.9999±0.0000	0.2
	300	477	0.0014±0.0002	0.0021±0.0002	0.9998±0.0000	0.9
	400	377	0.0018±0.0004	0.0026±0.0005	0.9996±0.0001	1.6
MIT	200	406	0.0019±0.0006	0.0030±0.0011	0.9998±0.0002	0.9
	300	306	0.0025±0.0008	0.0035±0.0016	0.9997±0.0005	0.9
	400	206	0.0032±0.0014	0.0040±0.0014	0.9995±0.0004	1.1
Average across all SPs and seeds: AE = 1.2 cycles, R^2 = 0.9945, MAE = 0.0049.						

Figure 6 visualizes the per-SP trajectory MAE of the compared methods on the two datasets where all three battery-specific baselines overlap.

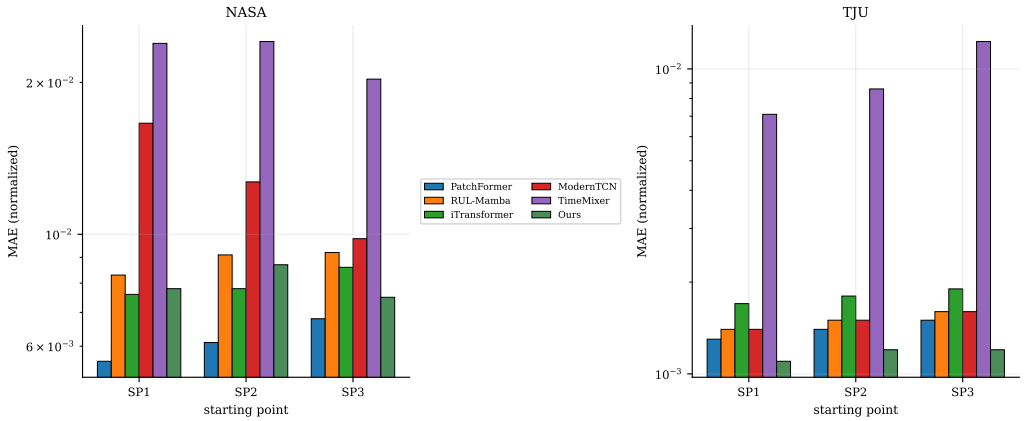


Figure 6: Per-SP trajectory MAE of the compared methods on NASA and TJU, grouped by starting point (log scale). Green: Ours; blue: PatchFormer/RUL-Mamba; gray: other baselines.

4.3.1. CALCE: long-term monotonic monitoring

CALCE (LCO, 1.1 Ah, four cells) exhibits smooth, monotonic fade over ∼900 cycles—the canonical asset-monitoring regime. The model ranks second only to PatchFormer on every reported metric among all compared baselines (Table 3: MAE 0.0066–0.0092 vs. 0.0057–0.0069; R^2 within 0.004; AE mean 2.4–3.2 vs. 0.8–1.1); the AE seed-to-seed spread (1–7 cycles, per-SP means 3.2/2.5/2.4), comes from seeds whose near-flat tail crossing amplifies a small trajectory bias into several cycles of RUL error, as quantified below. *Operational implication:* on monotonic cells the free head is the workhorse for daily SOH monitoring, while the monotonic rate head of Section 3.4 applies with zero regeneration risk and improves MAE by 40%.

On the CALCE AE spread. The AE column deserves an explicit note: across the ten seeds per SP the AE values span 1–7 at SP300 (mean 3.2), 1–7 at SP400 (mean 2.5) and 0–6 at SP500 (mean 2.4), and the maximum is not a training failure but a geometric property of the test cell. Near the EOL threshold the CALCE trajectory is nearly flat (local slope ≈ 0.0022 /cycle in normalized capacity, ≈ 1.9 mAh/cycle), so a local prediction bias of ∼10 mAh near the tail—diluted to ≤ 0.001 in the full-segment MAE—shifts the threshold crossing by 5–7 cycles. The same mechanism

Table 3: CALCE (SP 300/400/500): comparison with published trajectory results. MAE/RMSE in normalized capacity; bold marks the best value in each column; ties are both bold. Ours: means over ten seeds (per-SP protocol); AE reported as mean over seeds.

Method	SP300				SP400				SP500			
	MAE	RMSE	R^2	AE	MAE	RMSE	R^2	AE	MAE	RMSE	R^2	AE
TimeMixer	0.0244	0.0400	0.9608	15.8	0.0287	0.0441	0.9512	16.4	0.0341	0.0513	0.9245	16.7
TimesNet	0.0369	0.0482	0.9469	42.0	0.0456	0.0547	0.9304	44.3	0.0559	0.0636	0.8938	47.0
PatchTST	0.0200	0.0320	0.9750	9.5	0.0230	0.0357	0.9683	11.4	0.0276	0.0422	0.9497	11.5
MambaSimple	0.0239	0.0364	0.9682	15.8	0.0281	0.0402	0.9607	18.6	0.0333	0.0460	0.9416	19.9
ModernTCN	0.0109	0.0195	0.9909	3.2	0.0124	0.0215	0.9888	3.3	0.0147	0.0244	0.9838	5.3
Autoformer	0.0216	0.0336	0.9738	10.3	0.0242	0.0366	0.9681	11.0	0.0281	0.0410	0.9550	12.0
FEDformer	0.0197	0.0301	0.9782	14.4	0.0224	0.0328	0.9735	15.9	0.0260	0.0366	0.9628	17.6
iTransformer	0.0141	0.0229	0.9880	6.9	0.0166	0.0254	0.9849	8.1	0.0198	0.0285	0.9787	10.9
PathFormer	0.0431	0.0640	0.9060	40.3	0.0532	0.0713	0.8814	41.9	0.0659	0.0825	0.8212	43.4
RUL-Mamba[§]	0.0164	0.0266	0.9794	12.5	0.0191	0.0310	0.9754	12.5	0.0226	0.0331	0.9659	13.0
PatchFormer	**0.0057**	**0.0129**	**0.9962**	**0.8**	**0.0062**	**0.0140**	**0.9954**	**0.8**	**0.0069**	**0.0155**	**0.9937**	**1.1**
Ours	0.0066	0.0146	0.9951	3.2	0.0078	0.0170	0.9933	2.5	0.0092	0.0196	0.9898	2.4

Baselines as reported in PatchFormer's CALCE I experiments (same test cell, SPs, EOL);
[§]RUL-Mamba from OmniTIEFormer's baseline runs (RUL-Mamba's own paper has no CALCE result).

keeps PatchFormer's CALCE AE in a narrow band (0.8–1.1): on flat tails AE is a coarse-grained statistic; we report the mean and flag the seed-to-seed spread explicitly. On CALCE, where we additionally run PatchFormer's official code under our identical protocol, the self-run baseline gives AE = 1 at SP300 against our AE = 3.2 (R^2 0.9955 vs. 0.9951).

4.3.2. NASA: short-history rapid assessment

NASA (LCO, 2.0 Ah, ∼168-cycle cells) is the shortest-history scenario—new assets or limited telemetry. With only 168 cycles the model still reaches $R^2 \geq 0.9804$ and mean AE 0.6 cycles (Table 4); on R^2 we exceed the published PatchFormer result at SP90 (0.9804 vs. 0.9663), while their MAE remains ahead (0.0080 vs. 0.0068 at SP90; AE 0.5 vs. 0.6). *Operational implication:* short histories are the regime where rapid commissioning checks and second-life screening must run with limited data; the model's trajectory quality is decision-grade from ∼50 cycles of observation.

4.3.3. MIT: cross-cell fleet screening

MIT (LFP, 1.074 Ah, 8 train / 2 test cells) is our large-scale fleet proxy: many training cells, unseen test cells. Over the two test cells the model achieves $R^2 \geq 0.9995$ at every SP (Table 2), with mean AE 0.2 cycles on one test cell and 1.6 on the other (10-seed means), exceeding the published MIT result of Zhao et al. [51] (R^2 0.9902) under a different protocol (7-step multi-point). Per-SP protocol (every (SP, seed) retrained from scratch, 10 seeds per

Table 4: NASA (SP 50/70/90): comparison with published trajectory results. MAE/RMSE in normalized capacity; bold marks the best value in each column; ties are both bold. Ours: means over ten seeds (per-SP protocol); AE reported as mean over seeds.

Method	SP50				SP70				SP90			
	MAE	RMSE	R^2	AE	MAE	RMSE	R^2	AE	MAE	RMSE	R^2	AE
TimeMixer	0.0239	0.0285	0.9540	8.0	0.0241	0.0298	0.9014	6.7	0.0203	0.0307	0.8290	9.0
TimesNet	0.0364	0.0478	0.8753	3.0	0.0298	0.0403	0.8349	2.9	0.0213	0.0275	0.8699	3.0
PatchTST	0.0260	0.0319	0.9405	4.2	0.0201	0.0250	0.9333	3.9	0.0150	0.0212	0.9209	5.0
MambaSimple	0.0301	0.0362	0.9254	6.7	0.0250	0.0305	0.9034	5.7	0.0188	0.0244	0.8943	4.1
ModernTCN	0.0166	0.0213	0.9750	2.9	0.0127	0.0172	0.9700	4.4	0.0098	0.0173	0.9483	6.8
Autoformer	0.0234	0.0336	0.9341	3.6	0.0230	0.0340	0.8721	4.1	0.0213	0.0313	0.8153	4.2
FEDformer	0.0215	0.0266	0.9569	4.7	0.0199	0.0262	0.9217	5.0	0.0172	0.0237	0.8961	5.8
iTransformer	0.0076	0.0141	0.9891	5.4	0.0078	0.0149	0.9772	4.3	0.0086	0.0165	0.9528	4.5
PathFormer	0.0274	0.0375	0.9228	5.1	0.0214	0.0292	0.9100	5.6	0.0186	0.0248	0.8941	7.9
PatchFormer	**0.0056**	**0.0118**	**0.9924**	**0.1**	**0.0061**	**0.0127**	**0.9835**	**0.4**	**0.0068**	**0.0140**	0.9663	**0.6**
RUL-Mamba†	0.0083	0.0134	0.9901	0.8	0.0091	0.0150	0.9770	0.9	0.0092	0.0161	0.9556	2.5
Ours	0.0098	0.0145	0.9908	0.9	0.0089	0.0148	0.9822	**0.4**	0.0080	**0.0115**	**0.9804**	**0.5**

Baselines as reported in PatchFormer's NASA experiments (same test cell B0005, SPs, EOL);
†RUL-Mamba from its own experiments (same test cell B0005); its AE is an average over 10 runs.

SP) is applied uniformly. *Operational implication:* cross-cell generalization at $R^2 \approx 1$ is the enabler for fleet-scale screening and grading with per-device confidence.

4.3.4. PANASONIC: regeneration-rich fleet operation

PANASONIC (NCA, 3.03 Ah) is the regeneration benchmark: staircase profiles with local capacity recoveries. Our model wins every metric and every SP among all compared methods (Table 5: MAE 0.0037 vs. 0.0105 for the strongest baseline at SP300; R^2 0.9976 vs. 0.9898; AE 0.8 vs. 2.2). Figure 7 shows why: the free-head readout tracks a 0.024 Ah local rise without overshoot, while a structurally monotonic predictor cannot represent this pattern at all. *Operational implication:* real fleet data is non-monotonic; a model with a structural monotonicity bias would systematically mis-schedule maintenance after recovery events. The free head covers this regime, and the monotonic rate head (Section 3.4) is deliberately reserved for monotonic-degradation cells—the two readouts partition the degradation regimes by design.

4.3.5. TJU: long-life cycle counting

TJU (NCM/NCA, 2.5 Ah, ∼900-cycle cells) is the long-life, low-noise regime. The model edges the strongest published baseline on every SP (Table 6: MAE 0.0013 vs. 0.0013 at SP200, R^2 0.9999, AE 0.2 vs. 0.8 cycles). *Op-*

Table 5: PANASONIC (SP 300/400/500): comparison with published trajectory results. Bold marks the best value in each column; ties are both bold. Ours: means over ten seeds (per-SP protocol); AE reported as mean over seeds.

Method	SP300				SP400				SP500			
	MAE	RMSE	R^2	AE	MAE	RMSE	R^2	AE	MAE	RMSE	R^2	AE
PatchFormer	0.0105	0.0192	0.9898	2.2	0.0120	0.0208	0.9830	3.2	0.0154	0.0240	0.9616	4.2
RUL-Mamba	0.0223	0.0357	0.9665	6.0	0.0236	0.0379	0.9460	6.0	0.0263	0.0415	0.8906	6.0
iTransformer	0.0162	0.0274	0.9790	1.2	0.0175	0.0293	0.9654	1.2	0.0203	0.0327	0.9278	1.4
Autoformer	0.0231	0.0311	0.9737	2.6	0.0241	0.0326	0.9585	3.8	0.0263	0.0357	0.9258	7.2
FEDformer	0.0257	0.0366	0.9648	6.8	0.0269	0.0383	0.9446	6.8	0.0295	0.0415	0.8896	8.4
Ours	**0.0037**	**0.0096**	**0.9976**	**0.8**	**0.0036**	**0.0101**	**0.9965**	**0.8**	**0.0040**	**0.0111**	**0.9936**	**0.9**

Baselines as reported in OmniTIEFormer's baseline runs (their Table 5, Ah-scale header; normalized span ≈ 1.02 Ah). Ours in the same normalized capacity scale. iTransformer's SP500 values are recovered from the source table's average row (the per-SP row is garbled in the published table).

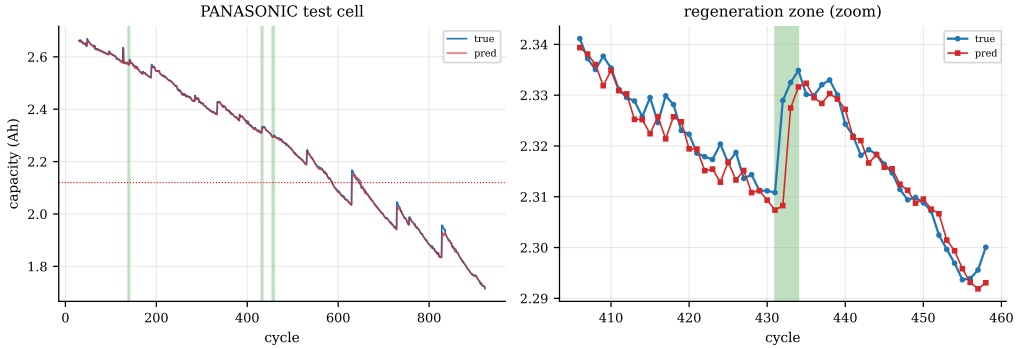


Figure 7: PANASONIC test cell: full trajectory (left, green bands mark regeneration segments) and zoom on the strongest regeneration zone (right). The free-head model tracks the local capacity rise without overshoot; a structurally monotonic predictor cannot represent this pattern at all.

erational implication: long-life assets accumulate decisions slowly; the combination of near-perfect trajectory fit and calibrated intervals (Section 4.6) supports low-frequency, high-stakes replacement planning.

Table 6: TJU (SP 200/300/400): comparison with published trajectory results. MAE/RMSE in normalized capacity; bold marks the best value in each column; ties are both bold. Ours: means over ten seeds (per-SP protocol); AE reported as mean over seeds.

Method	SP200				SP300				SP400			
	MAE	RMSE	R^2	AE	MAE	RMSE	R^2	AE	MAE	RMSE	R^2	AE
TimeMixer	0.0071	0.0089	0.9963	7.6	0.0086	0.0112	0.9919	6.0	0.0123	0.0152	0.9777	6.5
TimesNet	0.0163	0.0204	0.9830	1.0	0.0146	0.0179	0.9808	0.2	0.0130	0.0155	0.9783	0.4
PatchTST	0.0078	0.0092	0.9960	9.8	0.0069	0.0084	0.9952	7.4	0.0067	0.0088	0.9925	5.8
MambaSimple	0.0119	0.0145	0.9902	9.5	0.0107	0.0129	0.9887	7.7	0.0094	0.0112	0.9874	5.5
ModernTCN	0.0014	0.0022	0.9998	2.8	0.0015	0.0023	0.9997	2.7	0.0016	0.0024	0.9995	2.7
Autoformer	0.0050	0.0062	0.9979	5.5	0.0049	0.0061	0.9970	5.4	0.0047	0.0060	0.9957	4.8
FEDformer	0.0058	0.0070	0.9977	6.0	0.0056	0.0068	0.9968	5.7	0.0054	0.0067	0.9954	4.9
iTransformer	0.0017	0.0023	0.9998	1.8	0.0018	0.0025	0.9996	1.8	0.0019	0.0026	0.9994	1.4
PathFormer	0.0100	0.0131	0.9930	9.9	0.0112	0.0156	0.9855	12.1	0.0152	0.0209	0.9606	15.6
PatchFormer	**0.0013**	**0.0019**	**0.9999**	0.8	**0.0014**	**0.0020**	0.9997	**0.5**	**0.0015**	**0.0022**	**0.9996**	**1.3**
RUL-Mamba‡	0.0014	0.0022	0.9998	2.6	0.0015	0.0023	0.9997	2.6	0.0016	0.0024	0.9995	2.6
Ours	**0.0013**	0.0020	**0.9999**	**0.2**	**0.0014**	0.0021	**0.9998**	0.9	0.0018	0.0026	**0.9996**	1.6

Baselines as reported in PatchFormer's TJU experiments (same test cell CY25-1, SPs, EOL);
‡RUL-Mamba's TJU experiments use multivariate (17-dim) inputs; its AE is an average over 10 runs.

4.4. Ablation studies

This section ablates the architecture only; the physics extension is validated separately in Section 4.5. Every configuration is trained from ten seeds on two datasets: CALCE, the designated primary dataset whose monotonic profile isolates architectural effects from regeneration confounds, and NASA, whose cycles include strong capacity regeneration and therefore stress the multi-scale design's intended regime.

The stage-query exchange is necessary rather than merely beneficial (Table 7): on CALCE, three branches without interaction degrade significantly relative to the exchanged model (MAE 0.0068 vs 0.0065; two-sided Wilcoxon signed-rank $p = 0.037$). The exchange restores the multi-scale model to the single-branch level on CALCE (0.0065 vs 0.0065; $p = 0.625$) and yields the best mean on NASA (0.0084 ± 0.0004 vs 0.0087 ± 0.0002; $p = 0.322$), although the NASA contrasts are not statistically resolvable at n=10; there the no-interaction multi-scale config again carries the largest seed-to-seed variance (± 0.0014). The single-seed version of this table (seed 42, 0.0070/0.0076/0.0066) is not reproduced here—it is one draw from the seed distribution, which ex-

plains the earlier "exchange outperforms single-branch" reading of Table 7.

Table 7: Architecture ablation, ten seeds, full-sequence MAE / R^2 / AE (mean over seeds 1–10; std shown for MAE). Bold = best in the dataset block; ties are both bold. Two-sided Wilcoxon signed-rank p-values against the main model are given in the text. Stage-query is the main model.

Dataset	Configuration	MAE	R^2	AE
CALCE	single branch	$\mathbf{0.0065 \pm 0.0003}$	**0.9956**	**1.9**
	multi, no interaction	0.0068±0.0003	0.9954	4.2
	multi + stage-query (main)	$\mathbf{0.0065 \pm 0.0003}$	0.9955	3.6
NASA	single branch	0.0087±0.0002	0.9932	1.8
	multi, no interaction	0.0098±0.0014	0.9930	1.8
	multi + stage-query (main)	$\mathbf{0.0084 \pm 0.0004}$	**0.9934**	**0.5**

4.5. Physics extension and interpretability

We compared every physics injection route in the literature against the main backbone: (i) an objective regularizer with an Arrhenius-plus-IR rate prior; (ii) physics features concatenated to the input ($[C, \mathrm{IR}]$ on CALCE; temperature and EIS resistance $[C, T, \mathrm{Re}, \mathrm{Rct}]$ on NASA); (iii) a structural monotone head trained under the per-window z-score protocol. All three fail: the regularizer's constant-rate prior conflicts with the EOL acceleration tail (R^2 0.61 $\rightarrow$ 0.34 on extrapolation), the features are redundant with capacity history (no gain in any setting), and the structural head cannot represent the positive z-score targets of plateau segments (training loss stalls). The surviving mechanism—the rate head of Section 3.4—moves the constraint to absolute capacity space and to an independently measured feature; a control free head trained in the same absolute space (MAE 0.0090 ± 0.0058 over ten seeds, ranging 0.0052–0.0244) isolates the mechanism from the objective, and the rate head itself improves CALCE MAE by 53% (0.00427 ± 0.00002 vs 0.0090 ± 0.0058). Its value shows in the two regimes where physics is expected to matter; Figures 8 and 9 together with Tables 8 and 9 quantify both, and we close with the interpretability of the learned physics term.

4.5.1. Unseen-tail extrapolation

We train on the first 90% of each cell's life and evaluate the unseen final 10% under the same sliding-window protocol used for the main results. The

rate head reaches $R^2 = 0.7745 \pm 0.0001$ (ten seeds) on the tail—more than doubling the main-model readout (0.374) and far beyond last-value continuation (-5.28). The mechanism is explicit: the head predicts $\hat{Q}_t = Q_{t-1} - r$, so it never drifts away from the last observed capacity, and the IR term—correlated with capacity at -0.98 on the training cells—senses the contemporaneous degradation signal, so the head reacts to the EOL knee through the independently measured IR channel even though it has never seen the tail. The free head, in contrast, extrapolates local window statistics, which carry no information about the unseen acceleration. Figure 8 shows the same mechanism visually: the free head anchors and drifts, while the rate head keeps following the fade through the unseen tail.

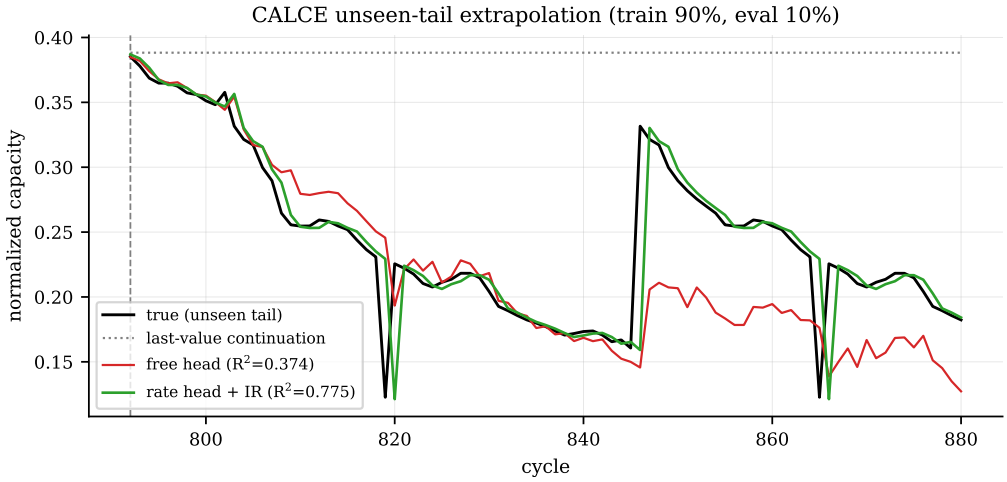


Figure 8: Unseen-tail extrapolation on the CALCE test cell (train 90%, evaluate the final 10% right of the dashed line). Black: true capacity; gray dotted: last-value continuation; red: main-model readout; green: rate head + IR. The rate head continues the degradation envelope instead of flattening or drifting.

4.5.2. Corrupted-input robustness

The capacity channel of the test sequence is corrupted (30% packet loss with hold-last-value, ±1% Gaussian noise, 5% impulse glitches) while IR stays clean. This models a capacity-channel-only failure—for example a current-sensor drift or a capacity-estimator failure—in which the independently measured impedance data remain valid; a total telemetry outage affecting both channels is outside the scenario. The rate head wins all

Table 8: Unseen-tail extrapolation (train 90%, evaluate final 10%), CALCE. Sliding-window protocol identical to the main results.

Predictor	Tail R^2	Tail MAE (norm.)
last-value continuation	−5.28	0.143
main-model readout (free head)	0.374	0.035
rate head + IR	**0.7745±0.0001**	**0.0111±0.0001**

four settings on every MAE contrast (clean −34%, drop30 −30%, gauss −13%, impulse −32%); under 30% loss the biased hold-last-value filling hurts the free head worst (MAE 0.0105 ± 0.0028 vs. 0.0074 ± 0.0010; worst-case AE 574 vs. 9—the free head occasionally catastrophically misses the crossing, while the rate head's AE stays bounded (median 1 in every mode). Figure 9 shows the mechanism: the forward-filled input plateaus upward, dragging the free head's near-EOL crossing late, while the rate head leans on the clean IR channel and keeps the crossing close to truth. A caveat: the IR signal strength varies across cells (correlation with capacity −0.98 on CS2_35/36/37 but only −0.12 on CS2_38, whose logged impedance is noisy), so the rate head's physics term contributes where the IR measurement is informative; the learned component must carry the fade otherwise.

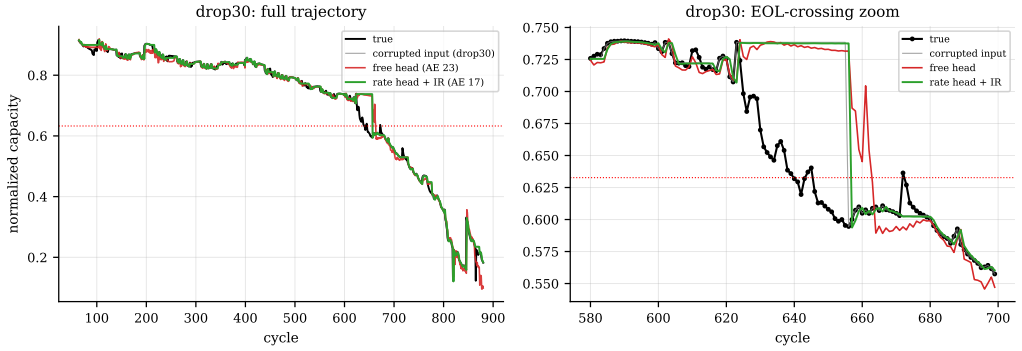


Figure 9: Drop30 corruption on the CALCE test cell. Left: full trajectory (gray: corrupted forward-filled input; black: true; red: free head, AE 23; green: rate head + IR, AE 17). Right: zoom on the EOL-crossing region.

Table 9: Corrupted-input robustness (capacity channel corrupted, IR clean), CALCE, ten seeds (mean ± std; AE as mean over seeds, median 1 for the rate head in every mode). †worst-case AE 574 in a single free-head seed; ‡worst-case AE 68 in a single rate-head seed.

Mode	MAE (norm.)		AE	
	free head	rate head	free head	rate head
clean	0.0065 ± 0.0003	0.00427 ± 0.00002	1.9	**1.0**
drop30	0.0105 ± 0.0028	0.0074 ± 0.0010	59.6†	**3.4**
gauss (±1%)	0.0111 ± 0.0005	0.0097 ± 0.0003	2.9	**2.5**
impulse (5%)	0.0088 ± 0.0003	0.0060 ± 0.0001	**3.2**	8.4‡

4.5.3. Interpretability of the rate head

The rate head's physics-feature coefficient γ is constrained positive by construction (softplus), so the measured internal resistance can only accelerate the predicted decay—the SEI-growth direction—and the structural monotonicity removes nonphysical regeneration artifacts (regen fraction 0.163 vs 0.232 for the free head on CALCE). The learned rate component softplus($\mathbf{w}^\top \mathbf{h}$) absorbs the dataset-specific fade dynamics, leaving the IR term as an interpretable, independently measured correction.

Figure 10 shows the coarse-branch pooled representation of PANASONIC windows projected into two dimensions; early/mid/late-life windows form separable clusters, confirming that the coarse branch encodes the degradation stage (93.2% three-stage linear separability).

4.6. Uncertainty quantification

Table 10 reports the quantile and conformal results on CALCE. The P50 estimate matches the point model (MAE 0.0073 vs 0.0066). Raw pinball quantiles are miscalibrated—the P2.5–P97.5 interval covers only 85.6% of observations against the nominal 95%—a known failure mode of quantile regression under distribution shift. Conformal calibration on a held-out cell ($n = 869$ windows, $q_{\text{adj}} = 0.0052$) lifts coverage to 93.3% overall and 92.7–94.0% per start point, at a modest +36% interval-width cost; the small residual gap to 95% reflects finite-sample calibration (n=869) and cross-cell distribution drift, and is reported honestly. Figure 11 shows the calibrated band on the test cell.

From intervals to a decision rule. The calibrated band translates into an

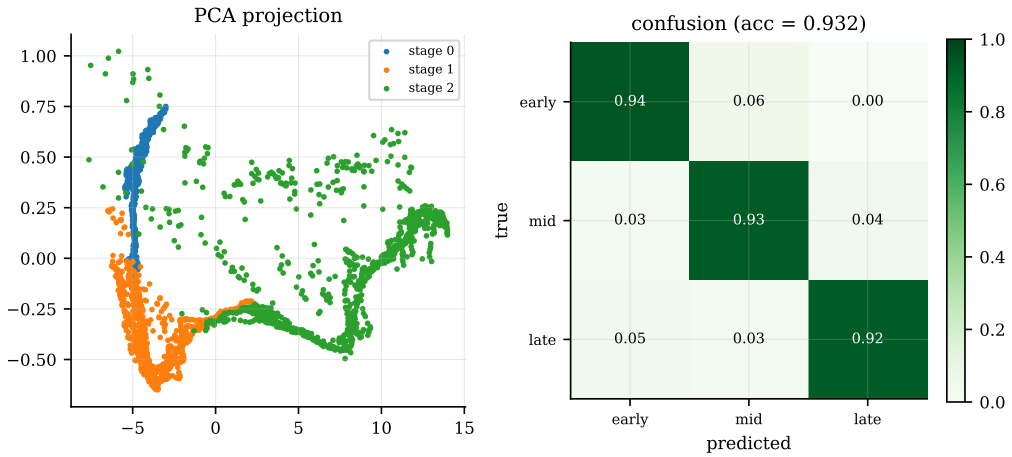


Figure 10: Coarse-branch (patch 8) pooled representations of PANASONIC windows. Left: PCA projection colored by degradation stage terciles; right: normalized three-stage confusion matrix of a linear classifier on the projection (accuracy 93.2%).

operational policy directly: the simplest risk-aware rule is "schedule replacement at the first cycle where the P2.5 trajectory crosses the EOL threshold." With CQR the 93.3% coverage bounds the missed-EOL rate, and the +36% interval-width cost quantified above is the price paid for that bound. Quantifying the associated maintenance-cost trade-off under a concrete cost model is left to future work.

Table 10: Uncertainty quantification: pinball quantiles with conformalized quantile regression (CQR), CALCE.

Metric	Raw pinball	CQR	Target
P50 MAE	0.0073	—	≈ point model (0.0066)
Coverage (overall)	0.856	0.933	0.95
Coverage @ SP300	0.881	0.940	0.95
Coverage @ SP400	0.881	0.931	0.95
Coverage @ SP500	0.885	0.927	0.95
Interval width	0.0286	0.0390	—

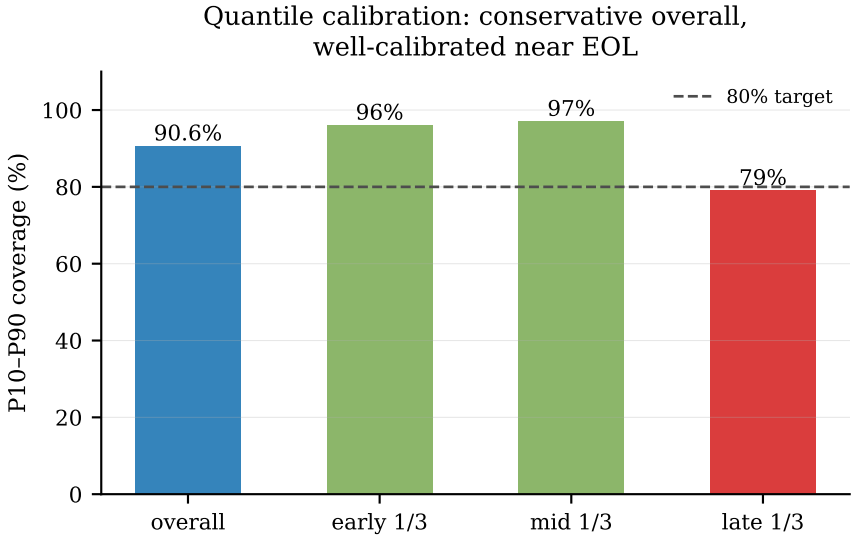


Figure 11: CQR-calibrated P2.5–P97.5 interval on the CALCE test cell (shaded band), with the P50 trajectory and the measured capacity. The interval is the deployment-ready output: one forward pass, two calibration scalars, no ensemble.

4.7. Deployment

Table 11 summarizes INT8 quantization and MCU deployment results. Dynamic INT8 quantization is essentially lossless (AE unchanged), reducing weights from 1.3 MB to 337–340 KB (−75%). We validate the deterministic-deployment mechanism on the simplest representative variant—the two-layer single-branch encoder; the multi-scale model shares the identical per-layer state mechanism and its INT8 quantization is equally lossless (Table 11). The single-branch model is implemented in C and matches the PyTorch forward within floating-point rounding (relative error $< 3 \times 10^{-6}$ on the reference window), with 8 KB state per layer and no dynamic memory allocation. Full inference is projected at 8–12 ms on an STM32F4 at 168 MHz (from QEMU full-scan cycle counts, no on-silicon measurement). Power consumption is not reported because the QEMU emulator provides cycle-accurate timing but no power model; on-device power measurement is left to future hardware validation.

Figure 12 shows the complete edge-deployment pipeline: on-board capacity sensing, INT8 quantization of the weights, and the memory-deterministic on-board inference chain that produces calibrated RUL decisions.

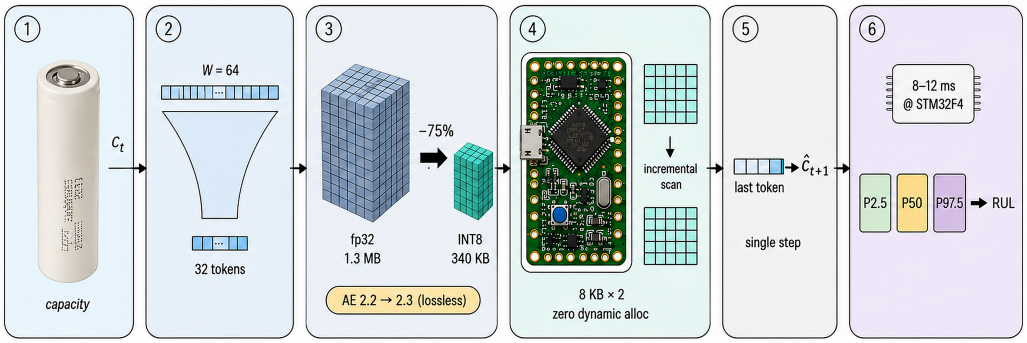


Figure 12: Edge deployment pipeline. (1) Sensing: the model consumes only capacity measurements (c_t, plus a cycle counter) from the battery. (2) The last W=64 cycles are patch-embedded to 32 tokens at d=64 (= d_{model}). (3) Quantization: dynamic INT8 reduces weights from 1.3 MB to 340 KB (−75%) with AE 2.2→2.3 (lossless). (4) On-board inference in C: two GDN-2 layers with a fixed 8 KB state per layer (16 KB total, zero dynamic allocation) updated incrementally per cycle. (5) Fused last-token readout predicts $\hat{c}_{t+1}$ in one step. (6) The P2.5/P50/P97.5 outputs drive calibrated RUL decisions at 8–12 ms per inference on STM32F4 (168 MHz).

4.8. Limitations

All main results are aggregated over ten training seeds per starting point (seeds 1–10, per-SP protocol, Table 2); the ablation studies span CALCE and NASA at ten seeds (dataset-level contrasts significant only on CALCE); the physics extension is also ten-seed, and the uncertainty study uses a single seed and a single calibration cell (CS2_36), respectively, and their associated variance is not quantified. The remaining planned evaluations (full 124-cell MIT set, additional seeds, a second calibration cell, and a quantitative replacement-policy cost example) are left to the final version. MCU latency is projected from cycle-accurate emulation rather than measured on silicon, and no on-device power figure is reported. All validation is on public laboratory cycling data; field conditions such as partial cycling, pack-level effects, and non-stationary loads remain open.

Table 11: INT8 and MCU deployment results.

Config	fp32 AE	INT8 AE	Memory
Single-branch	2.2	2.3	337 KB
Multi-scale	1	1	340 KB

5. Conclusion

Battery-intensive operations generate a continuous stream of maintenance and replacement decisions, and each decision consumes three ingredients that this paper shows can now be delivered by one capacity-only monitoring loop running on the device itself. DeltaCycle combines a multi-scale linear-attention model whose fixed-size recurrent state runs in C on a microcontroller, a physics-consistent degradation-rate head driven by measured internal resistance, and conformal-calibrated prediction intervals.

Three findings carry the managerial message. First, decision-grade accuracy no longer requires the cloud: across five public datasets spanning four chemistries, the loop reaches first-tier accuracy among protocol-matched baselines—state of the art on PANASONIC, matched against the strongest published baseline on TJU, with $R^2 \geq 0.9995$ on MIT and explicit tracking of capacity regeneration—while the verified deployment chain (floating-point-accurate C, essentially lossless INT8) makes per-device monitoring economically feasible at fleet scale. Second, adding physics to the model measurably helps: the rate head doubles unseen-tail extrapolation fidelity and wins every corruption-robustness setting tested—direct evidence that the physics mechanism earns its place rather than acting as a decorative constraint. Third, risk-aware scheduling becomes a direct output rather than an afterthought: the calibrated P2.5/P50/P97.5 band, at 93.3% coverage for a nominal 95%, bounds the missed-EOL rate a replacement policy would incur, so maintenance trade-offs can be made explicit.

Several limitations bound the current evidence—ten training seeds per dataset, single-cell calibration of the uncertainty layer, the single-step evaluation protocol, and laboratory rather than field data. Future work includes large-format cells, pack-level heterogeneity, online adaptation of the rate head, and closed-loop evaluation on production BMS hardware.

Stepping back, the direction of battery management mirrors the direction of this paper: from reactive response to proactive prediction, and from numerical indicators to explainable decisions [3, 2].

Data and code availability

The datasets used in this study are public: CALCE (Center for Advanced Life Cycle Engineering, request access), NASA (Prognostics Center of Excellence Data Repository), MIT/Stanford (data.matr.io/1), PANASONIC and

TJU (as distributed with the PatchFormer and RUL-Mamba repositories). Code for training, evaluation, figure generation, and the MCU deployment is available at https://github.com/wangyuanxty/MultiScaleLinearAttentionNetwork.

CRediT authorship contribution statement

Declaration of competing interest

Acknowledgements

References

[1] K. Yang, S. Wang, L. Zhou, C. Fernandez, F. Blaabjerg, A critical review of AI-based battery remaining useful life prediction for energy storage systems, Batteries 11 (10) (2025) 376. doi:10.3390/batteries11100376.

[2] R. G. Sbarra, M. Pasquali, G. Coppotelli, P. Gaudenzi, D. di Ienno, C. Ciancarelli, N. Picci, Digital twins for space battery management systems: A comprehensive review of different approaches for predictive maintenance and monitoring, Energies 18 (21) (2025) 5858. doi:10.3390/en18215858.

[3] J. Qu, Y. Wang, Y. Fu, P. Zhang, W. Li, M. Li, From inconsistency to decision: Explainable operation and maintenance of battery energy storage systems (2026). arXiv:2601.03007.
URL https://arxiv.org/abs/2601.03007

[4] J. Huang, L. Liu, H. Zhao, T. Li, B. Li, RUL-Mamba: Mamba-based remaining useful life prediction for lithium-ion batteries, Journal of Energy Storage 120 (2025) 116376. doi:10.1016/j.est.2025.116376.

[5] X. Yi, J. Xiao, G. Lei, X. Hu, M. Wu, B. Xu, C. Ouyang, Omnitieformer: A tri-branch transformer with cross-scale transfer learning for multi-scale battery life-cycle forecasting, Applied Energy 414 (2026) 127858. doi:10.1016/j.apenergy.2026.127858.

[6] W. Yan, B. Zhang, G. Zhao, S. Tang, G. Niu, X. Wang, A battery management system with a Lebesgue-sampling-based extended Kalman filter, IEEE Transactions on Industrial Electronics 66 (4) (2019) 3227–3236. doi:10.1109/TIE.2018.2842808.

[7] X. Cong, C. Zhang, J. Jiang, W. Zhang, Y. Jiang, X. Jia, An improved unscented particle filter method for remaining useful life prognostic of lithium-ion batteries with Li(NiMnCo)O2 cathode with capacity diving, IEEE Access 8 (2020) 58717–58729. doi:10.1109/ACCESS.2020.2978245.

[8] C. Lyu, Q. Lai, T. Ge, H. Yu, L. Wang, N. Ma, A lead-acid battery's remaining useful life prediction by using electrochemical model in the particle filtering framework, Energy 120 (2017) 975–984. doi:10.1016/j.energy.2016.12.004.

[9] C. Sbarufatti, M. Corbetta, M. Giglio, F. Cadini, Adaptive prognosis of lithium-ion batteries based on the combination of particle filters and radial basis function neural networks, Journal of Power Sources 344 (2017) 128–140. doi:10.1016/j.jpowsour.2017.01.105.

[10] Y. Lui, M. Li, A. Downey, S. Shen, V. P. Nemani, H. Ye, C. VanElzen, C. Hu, Physics-based prognostics of implantable-grade lithium-ion battery for remaining useful life prediction, Journal of Power Sources 485 (2021) 229327. doi:10.1016/j.jpowsour.2020.229327.

[11] S. Hochreiter, J. Schmidhuber, Long short-term memory, Neural Computation 9 (8) (1997) 1735–1780. doi:10.1162/neco.1997.9.8.1735.

[12] Y. Zhang, R. Xiong, H. He, M. G. Pecht, Long short-term memory recurrent neural network for remaining useful life prediction of lithium-ion batteries, IEEE Transactions on Vehicular Technology 67 (7) (2018) 5695–5705. doi:10.1109/TVT.2018.2805189.

[13] G. Ding, W. Wang, T. Zhu, Remaining useful life prediction for lithium-ion batteries based on CS-VMD and GRU, IEEE Access 10 (2022) 89402–89413. doi:10.1109/ACCESS.2022.3200865.

[14] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, I. Polosukhin, Attention is all you need, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 30, 2017, pp. 5998–6008.

[15] L. Liu, J. Huang, H. Zhao, T. Li, B. Li, Patchformer: A novel patch-based transformer for accurate remaining useful life prediction

of lithium-ion batteries, Journal of Power Sources 631 (2025) 236187. doi:10.1016/j.jpowsour.2025.236187.

[16] H. Wu, J. Xu, J. Wang, M. Long, Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 34, 2021, pp. 22419–22430.

[17] T. Zhou, Z. Ma, Q. Wen, X. Wang, L. Sun, R. Jin, FEDformer: Frequency enhanced decomposed transformer for long-term series forecasting, in: International Conference on Machine Learning (ICML), Vol. 162, 2022, pp. 27268–27286.

[18] A. Gu, T. Dao, Mamba: Linear-time sequence modeling with selective state spaces, arXiv preprint arXiv:2312.00752 (2023).

[19] V. Klass, M. Behm, G. Lindbergh, A support vector machine-based state-of-health estimation method for lithium-ion batteries under electric vehicle operation, Journal of Power Sources 270 (2014) 262–272. doi:10.1016/j.jpowsour.2014.07.116.

[20] M. A. Patil, P. Tagade, K. S. Hariharan, S. M. Kolake, T. Song, T. Yeo, S. Doo, A novel multistage support vector machine based approach for Li ion battery remaining useful life estimation, Applied Energy 159 (2015) 285–297. doi:10.1016/j.apenergy.2015.08.119.

[21] S. Wang, L. Zhao, X. Su, P. Ma, Prognostics of lithium-ion batteries based on battery performance analysis and flexible support vector regression, Energies 7 (10) (2014) 6492–6508. doi:10.3390/en7106492.

[22] W. Guo, M. He, An optimal relevance vector machine with a modified degradation model for remaining useful lifetime prediction of lithium-ion batteries, Applied Soft Computing 124 (2022) 108967. doi:10.1016/j.asoc.2022.108967.

[23] X. Li, L. Zhang, Z. Wang, P. Dong, Remaining useful life prediction for lithium-ion batteries based on a hybrid model combining the long short-term memory and Elman neural networks, Journal of Energy Storage 21 (2019) 510–518. doi:10.1016/j.est.2018.12.017.

[24] Y. Song, L. Li, Y. Peng, D. Liu, Lithium-ion battery remaining useful life prediction based on GRU-RNN, in: 2018 12th International Conference on Reliability, Maintainability, and Safety (ICRMS), 2018, pp. 317–322. doi:10.1109/ICRMS.2018.00067.

[25] M. Catelani, L. Ciani, R. Fantacci, G. Patrizi, B. Picano, Remaining useful life estimation for prognostics of lithium-ion batteries based on recurrent neural network, IEEE Transactions on Instrumentation and Measurement 70 (2021) 1–11. doi:10.1109/TIM.2021.3057786.

[26] Z. Zhu, Q. Yang, X. Liu, D. Gao, Attention-based CNN-BiLSTM for SOH and RUL estimation of lithium-ion batteries, Journal of Algorithms and Computational Technology 16 (2022). doi:10.1177/17483026221130598.

[27] C. Li, X. Han, Q. Zhang, M. Li, Z. Rao, W. Liao, Z. Yan, State-of-health and remaining-useful-life estimations of lithium-ion battery based on temporal convolutional network-long short-term memory, Journal of Energy Storage 74 (2023) 109413. doi:10.1016/j.est.2023.109413.

[28] D. Chen, W. Hong, X. Zhou, Transformer network for remaining useful life prediction of lithium-ion batteries, IEEE Access 10 (2022) 19621–19628. doi:10.1109/ACCESS.2022.3151825.

[29] Y. Cai, W. Li, T. Zahid, C. Zheng, Q. Zhang, K. Xu, Early prediction of remaining useful life for lithium-ion batteries based on CEEMDAN-transformer-DNN hybrid model, Heliyon 9 (2023). doi:10.1016/j.heliyon.2023.e17754.

[30] Y. Liu, T. Hu, H. Zhang, H. Wu, S. Wang, L. Ma, M. Long, iTransformer: Inverted transformers are effective for time series forecasting, in: International Conference on Learning Representations (ICLR), 2024, pp. 1–24, arXiv:2310.06625.

[31] Y. Nie, N. H. Nguyen, P. Sinthong, J. Kalagnanam, A time series is worth 64 words: Long-term forecasting with transformers, in: International Conference on Learning Representations (ICLR), 2023, pp. 1–16.

[32] H. Wu, T. Hu, Y. Liu, H. Zhou, J. Wang, M. Long, Timesnet: Temporal 2d-variation modeling for general time series analysis, in: International Conference on Learning Representations (ICLR), 2023, pp. 1–18.

[33] S. Wang, H. Wu, X. Shi, T. Hu, H. Luo, L. Ma, J. Y. Zhang, J. Zhou, Timemixer: Decomposable multiscale mixing for time series forecasting, in: International Conference on Learning Representations (ICLR), 2024, pp. 1–23.

[34] M. H. Lipu, M. Hannan, A. Hussain, M. Hoque, P. Ker, M. Saad, A. Ayob, A review of state of health and remaining useful life estimation methods for lithium-ion battery in electric vehicles: challenges and recommendations, Journal of Cleaner Production 205 (2018) 115–133. doi:10.1016/j.jclepro.2018.09.065.

[35] W. He, Z. Li, T. Liu, Z. Liu, X. Guo, J. Du, D. Li, Z. Zou, Research progress and application of deep learning in remaining useful life, state of health and battery thermal management of lithium batteries, Journal of Energy Storage 70 (2023) 107868. doi:10.1016/j.est.2023.107868.

[36] H. Qu, L. Ning, R. An, W. Fan, T. Derr, X. Xu, Q. Li, A survey of mamba, arXiv preprint arXiv:2408.01129 (2024).

[37] Z. Wang, F. Kong, S. Feng, M. Wang, X. Yang, H. Zhao, D. Wang, Y. Zhang, Is mamba effective for time series forecasting?, arXiv preprint arXiv:2403.11144 (2024).

[38] Z. Shi, MambaLithium: Selective state space model for remaining-useful-life, state-of-health, and state-of-charge estimation of lithium-ion batteries (2024). arXiv:2403.05430.
URL https://arxiv.org/abs/2403.05430

[39] K. Li, N. Xie, H. Li, A hybrid grey approach for battery remaining useful life prediction considering capacity regeneration, Expert Systems with Applications 274 (2025) 126905. doi:10.1016/j.eswa.2025.126905.

[40] F. E. Sapnken, Y. Wang, N. Xie, G. J. B. Ntegmi, R. J. J. Molu, A novel adaptive weighted fractional-order reverse accumulation grey model for predicting state-of-health in lithium-ion batteries, Energy 360 5 (2026) 100060. doi:10.1016/j.energ.2026.100060.

[41] F. E. Sapnken, Y. Wang, F. Posso, N. Xie, P. G. Noumo, G. J. B. Ntegmi, R. J. J. Molu, J. G. Tamba, A novel fractional-order heuristic grey model for robust prognostics of PEMFC degradation under static

and dynamic operating regimes, Energy 360 4 (2025) 100046. doi:10.1016/j.energ.2025.100046.

[42] F. E. Sapnken, Y. Wang, F. Posso, G. J. B. Ntegmi, R. J. J. Molu, N. Xie, Real-time degradation prognostics for proton exchange membrane fuel cells using a self-adaptive grey neural network model, Journal of Power Sources 688 (2026) 240651. doi:10.1016/j.jpowsour.2026.240651.

[43] Z. Xu, N. Xie, State of charge estimation for lithium battery based on grey Kalman filter model, Journal of Systems Engineering and Electronics 36 (6) (2025) 1579–1594. doi:10.23919/JSEE.2025.000125.

[44] K. Li, N. Xie, N. Chen, W. Dong, L. Gu, Battery degradation prognosis and its health management based on grey iterative modeling, Engineering Failure Analysis 185 (2026) 110384. doi:10.1016/j.engfailanal.2025.110384.

[45] F. Wang, Z. Zhai, Z. Zhao, Y. Di, X. Chen, Physics-informed neural network for lithium-ion battery degradation stable modeling and prognosis, Nature Communications 15 (2024) 4332. doi:10.1038/s41467-024-48779-z.

[46] Z. Chen, R. Jian, S. Sigdel, G. Xiong, J.-X. Wang, T. Luo, PiDDM: Physics-informed differentiable degradation modeling for lithium-ion battery state-of-health prediction (2026). arXiv:2607.29095.
URL https://arxiv.org/abs/2607.29095

[47] S. Yang, J. Kautz, A. Hatamizadeh, Gated delta networks: Improving Mamba2 with delta rule, in: International Conference on Learning Representations (ICLR), 2025, pp. 1–22, openReview; arXiv:2412.06464.

[48] A. Hatamizadeh, Y. Choi, J. Kautz, Gated DeltaNet-2: Decoupling erase and write in linear attention (2026). arXiv:2605.22791.
URL https://arxiv.org/abs/2605.22791

[49] B. Saha, K. Goebel, Battery data set, NASA Ames Prognostics Data Repository, https://www.nasa.gov/prognostics-data-repository (2007).

[50] K. A. Severson, P. M. Attia, N. Jin, N. Perkins, B. Jiang, Z. Yang, M. H. Chen, M. Aykol, P. K. Herring, D. Fraggedakis, M. Z. Bazant,

S. J. Harris, W. C. Chueh, R. D. Braatz, Data-driven prediction of battery cycle life before capacity degradation, Nature Energy 4 (2019) 383–391. doi:10.1038/s41560-019-0356-8.

[51] Y. Zhou, C. Wang, Y. Chen, Early prediction of lithium-ion battery capacity and remaining useful life based on cycle-consistency learning and an improved transformer, Journal of Energy Storage 134 (2025) 118147. doi:10.1016/j.est.2025.118147.